

Project 11 (Bonus) — MCP Crew Multi-Agent Research Team Powered By MCP Servers

Title: MCP Crew Multi Agent Research Team Powered By MCP Servers

Type: Multi-agent (4 agents: 1 Supervisor + 3 specialists) · **Stack:** Python, LangGraph, `Command` tool, `mcp` (the official MCP Python SDK), OpenAI SDK, python-dotenv, pydantic, asyncio · **Level:** Advanced **Tagline:** A supervisor-led team of three specialists — Notes, Web, and Writer — where each specialist gets its own tools from its own dedicated MCP server, not from one shared Python toolbox.

Overview

MCP Crew is a Supervisor-led team of three agents — a Notes Agent, a Web Agent, and a Writer — built to show what happens when a coordinated multi-agent system gets its tools from separate, independently-owned MCP servers instead of one shared Python toolbox. The Notes Agent and Web Agent each open their own connection to their own dedicated MCP server and discover their tools at runtime; the Writer synthesizes both specialists' findings into one coherent report. A `Command`-based Supervisor routes work between them and keeps the team working, with a clearly-flagged gap in the final report, even when one specialist's MCP server is unreachable. This mirrors how real organizations split ownership across teams and services, where one team's outage shouldn't silently break every capability a user-facing assistant depends on.

Features

- Routes work between three specialist agents using a `Command`-based Supervisor
- Notes Agent and Web Agent each connect to their own dedicated MCP server and discover tools at runtime, instead of hardcoding them
- Writer agent synthesizes multiple specialists' findings into one coherent report instead of concatenating them
- Each specialist is proven to work standalone, MCP connection and all, before ever being wired into the team
- Degrades gracefully when a specialist's MCP server is unreachable, still finishing with a clearly-flagged gap instead of crashing
- Shared state holds only each specialist's finished result, never its private tool-call scratchpad

Tech Stack

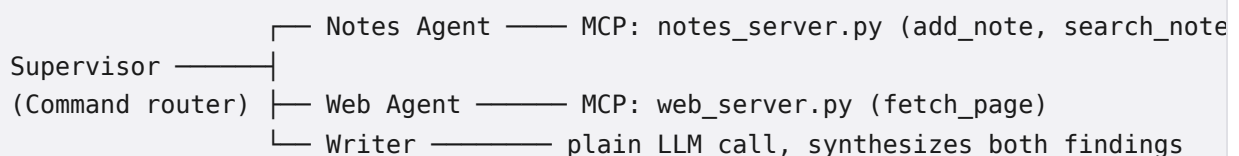
- Python
- LangGraph
- `Command` tool (agent routing)
- `mcp` (the official MCP Python SDK)
- OpenAI SDK
- `python-dotenv`
- `pydantic`
- `asyncio`

Prerequisites

- Understand how an MCP server exposes tools, and how a client discovers and calls them at runtime
- Familiar with the MCP handshake, stdio transport, and the `ClientSession` / `stdio_client` connection pattern
- Comfortable with multi-agent coordination, including Supervisor routing with `Command`
- Some familiarity with Python's `async` / `await`, since MCP client code is `asyncio`-based

Architecture

A `Command`-based Supervisor sits at the entry point of a LangGraph graph and routes each request to whichever specialist(s) it needs. The Notes Agent opens its own MCP client connection to a dedicated notes server (`add_note` , `search_notes` tools, in-memory backed) and the Web Agent opens its own connection to a separate web server (a `fetch_page` tool). Neither specialist's MCP connection is visible to the Supervisor — it only calls each specialist as a graph node and reads back its finished result from shared state. Once the specialists a request needs have contributed, the Supervisor routes to a Writer agent, a plain LLM call with no tools, which combines their findings into one report. If the Web Agent's server is unreachable, a timeout and `try / except` inside its own node catches the failure, sets a `web_unavailable` flag in shared state, and the Writer's prompt reads that flag to say so explicitly in the final report.



Setup (do this once, before Step 1)

```
cd project_11_mcpcrew
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install mcp openai langchain langchain-openai langgraph python-dotenv pydantic
pip freeze > requirements.txt
```

Same `.env` setup as every other project:

```
OPENAI_API_KEY=sk-...
LOG_LEVEL=INFO
```

And the same `.env.example` with the key names but no real secret.

This project is `asyncio`-based wherever it talks to an MCP server, the same as any MCP client code. If `async / await` is still unfamiliar, a quick review of Python's `async/await` basics is worth doing first — but the minimal client/server example already shows the exact pattern (`asyncio.run(main())` around an `async def main()`) you'll be reusing here. You do **not** need an existing MCP server running elsewhere — Step 1 has you build a small, self-contained notes server of your own inside this folder, so this project doesn't depend on another one existing first.

Troubleshooting

Problem	Fix
The Supervisor tries to call a specialist's tools directly, instead of letting the specialist own its own MCP connection	Keep the boundary clean: the Supervisor only ever calls a specialist <i>node function</i> and reads its result from shared state — it never touches <code>ClientSession</code> or <code>stdio_client</code> itself, exactly like a Supervisor should never call a tool directly, only route to the agent that owns it
The Writer's report reads like two disconnected paragraphs stapled together	Give the Writer's prompt both specialists' findings <i>and</i> an explicit instruction to synthesize, not concatenate
A specialist's MCP connection opens and closes correctly in Step 1's standalone test, but hangs once it's a graph node	Check whether your graph node is <code>async</code> and being invoked with <code>.ainvoke()</code> / <code>astream()</code> , or whether you're wrapping the same <code>async def</code> in <code>asyncio.run()</code> inside a plain sync node — pick one pattern and use it consistently across every specialist node
The Web Agent's server is down, and the whole graph crashes instead of degrading	Wrap the connection attempt in <code>asyncio.wait_for(...)</code> inside <code>try / except</code> , catch it <i>inside</i> the Web Agent's own node function, and write a clear "unavailable" result into shared state instead of letting the exception escape the node

Problem	Fix
The final report doesn't mention that a specialist's data is missing, even though degradation worked correctly	The degradation flag has to actually reach the Writer's prompt — check your shared state includes something like <code>web_unavailable: bool</code> , and that the Writer's prompt-building code reads it and says so explicitly, instead of just silently having an empty <code>web_findings</code> field

Usage Example

Use this scenario, or one close to it, for every step below:

Scenario: MCP-Crew is asked to put together a short status report on a made-up internal project, "Project Atlas." It has two sources: a **notes server** holding a handful of `.md`-style notes someone already wrote about Atlas, and a **web server** that can "fetch" a couple of known internal status pages (mocked — see Step 1).

Test these tasks (save them in `test_tasks.py` so you reuse the same ones every run): 1. "What do my notes say about Project Atlas?" — needs only the Notes Agent. 2. "Fetch the Project Atlas status page and summarize what it says." — needs only the Web Agent. 3. "Check my notes for anything about Project Atlas, fetch its status page too, and give me one combined report." — needs **both** specialists, then the Writer — this is the case that actually proves the Supervisor's multi-step routing and the Writer's synthesis both work, not just that each specialist works alone. 4. Task 3 again, but with the **Web server deliberately not started** — this is Step 4's test: the team should still produce a report from the Notes Agent's findings alone, with the report clearly saying the web status page couldn't be reached, not a crash and not a report that silently pretends the web check happened.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

How To Build This — Step by Step

Step 1 — Two Standalone MCP-Connected Specialists

Project: MCP-Crew-Multi-Agent-Research-Team-Powered-By-MCP-Servers — Step 1 of 4: Two Standalone MCP-Connected Specialists

What this step does: builds the Notes Agent and the Web Agent, each connecting to its own small MCP server, each one proven to work completely alone — no Supervisor, no LangGraph, no teammate exists yet. **Why this step matters:** this is the same "prove the connection before adding complexity" instinct any solid build starts with. If a specialist's MCP connection is broken, debugging that *inside* Step 2's Supervisor graph means debugging two things at once — the connection and the routing — instead of one at a time. **When you'll hit this for real:** the first time you wire any external service into a multi-agent system — you always prove the one connection works standalone before

trusting a coordinator to call it correctly. **Helpful background:** how MCP servers and clients actually connect; MCP's three building blocks — Tools, Resources, and Prompts.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `notes_server.py`: a `MCPServer` server (a simplified version of a standalone MCP tool server — you don't need SQLite or a Resource/Prompt here, just the two Tools) exposing `add_note(text: str) -> str` and `search_notes(query: str) -> list[str]`, backed by a plain in-memory list, seeded with 3-4 made-up notes at startup so `search_notes` has something real to find. 2. Write `web_server.py`: a `MCPServer` server exposing exactly one tool, `fetch_page(url: str) -> str`. Make this a **mocked, stubbed fetch** — a small Python dict mapping 2-3 known URLs (like `"https://intranet.example.com/atlas/status"`) to canned page text, and a clear `"No content available for this URL."` string for anything else. Write a comment at the top of this file saying plainly that a real deployment would connect this tool to a real fetch/search MCP server (or the public reference fetch server) instead of a hardcoded dict — this project mocks it on purpose, for reliable, repeatable teaching. 3. Write `notes_agent.py`: `run_notes_agent(task: str) -> str`, which opens its own connection to `notes_server.py` (the same `stdio_client / ClientSession / initialize()` shape used for any standalone MCP client), discovers its tools, and runs a small ReAct-style loop (the same tool-calling loop pattern used for MCP-connected agents) to answer the task using `search_notes` and/or `add_note`. 4. Write `web_agent.py`: `run_web_agent(task: str) -> str`, the same shape, connecting to `web_server.py` and using `fetch_page`. 5. Write `main.py` that calls `run_notes_agent` and `run_web_agent` one after another, on tasks 1 and 2 from **A Real Example**, and prints both results — confirm each specialist works completely on its own before Step 2 puts a Supervisor in front of them.

Your files after Step 1:

```
project_11_mcpcrew_multi_agent_mcp/
├── .env / .env.example
├── config.py                (standard config/logging setup)
├── logging_setup.py         (standard config/logging setup)
├── notes_server.py          → MCPServer: add_note(text), search_notes(query) –
├── web_server.py            → MCPServer: fetch_page(url) – mocked/stubbed con
├── mcp_connection.py        → connect_stdio_server(command, args) -> Client
├── notes_agent.py           → run_notes_agent(task) -> str, standalone, c
├── web_agent.py             → run_web_agent(task) -> str, standalone, co
└── main.py                  → runs both specialists standalone, no sup
```

Step 2 — A Supervisor Routes Between Them

Project: MCP Crew Multi-Agent Research Team Powered By MCP Servers — Step 2 of 4: A Supervisor Routes Between Them

What this step does: wraps Step 1's two proven specialists in a real LangGraph graph, and adds a `Command`-based Supervisor (the same Supervisor routing pattern used elsewhere) that decides, per request, whether to call the Notes Agent, the Web Agent, or both, in sequence. No Writer yet — whichever specialist(s) ran, their raw result is the graph's output. **Why this step matters:** this is the actual multi-agent part of the project — proving a Supervisor can route correctly to specialists whose tools it knows nothing about, since each specialist's MCP connection is fully private to that specialist's own node. **What's new vs. Step 1:** a shared `TeamState`, a Supervisor node using `Command`, and Step 1's two specialists rewired as graph nodes instead of functions called directly by `main.py`. **What stays the same:** the specialists' own internal logic — the MCP connection, tool discovery, and ReAct loop inside `notes_agent.py` and `web_agent.py` — is unchanged from Step 1. You're wrapping them, not rewriting them. **When you'll hit this for real:** any time a working single-purpose agent needs to join a bigger, coordinated system — the same moment any earlier team's specialists were each proved individually before a Supervisor was added to route between them. **Helpful background:** the `Command` tool for handing work between agents; shared state design — what belongs in shared state, and what doesn't.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Design `state.py` : a `TeamState(TypedDict)` with `task: str`, `notes_findings: str`, `web_findings: str`, and routing info like `next_agent: str`. Follow the standard multi-agent shared-state rule directly — only each specialist's *finished* result goes in shared state; each specialist's own scratchpad (its ReAct loop's intermediate steps) stays private inside that specialist's own node function. 2. Rewrite `notes_agent.py` and `web_agent.py` to expose a graph-node function too — `notes_agent_node(state: TeamState) -> Command`, `web_agent_node(state: TeamState) -> Command` — each one calls its existing Step 1 function internally, writes the result into the right shared-state field, and returns a `Command` sending control back to the Supervisor. 3. Write `supervisor.py` : a node that looks at `state["task"]` (and what's already been filled in) and returns a `Command` routing to `"notes_agent"`, `"web_agent"`, or `"FINISH"` — for task 3 from **A Real Example**, this means routing to one specialist, seeing its result land in state, then routing to the other, before finishing. 4. Write `graph.py` : a `StateGraph(TeamState)` wiring the Supervisor and both specialist nodes together, with the Supervisor as the entry point. 5. Update `main.py` to build and run the graph on tasks 1, 2, and 3 from **A Real Example** — confirm task 3's final state has *both* `notes_findings` and `web_findings` filled in, proving the Supervisor really called both specialists in one run, not just the first one it happened to pick.

Your files after Step 2:

```
project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py
```

```

├─ state.py           → TeamState(TypedDict): task, notes_findings, web_fin
├─ supervisor.py      → Command-based router: notes_agent / web_agent / F
├─ notes_agent.py     → adds notes_agent_node(state) -> Command, wraps
├─ web_agent.py       → adds web_agent_node(state) -> Command, wraps
├─ graph.py           → StateGraph wiring supervisor + both special
└─ main.py            → runs the graph, prints whichever specialis

```

Step 3 — The Writer Agent Synthesizes a Final Report

Project: MCP Crew Multi-Agent Research Team Powered By MCP Servers — Step 3 of 4: The Writer Agent Synthesizes a Final Report

What this step does: adds the Writer — a plain LLM call, no MCP server, no tools at all — which the Supervisor calls last, once the specialists it needed have already contributed. It turns "here's what 2 agents found separately" into one coherent report. **Why this step matters:** two specialists' raw findings sitting side by side in state isn't a report — it's two paragraphs a person still has to stitch together themselves. The Writer is the piece that actually produces the thing the user asked for.

What's new vs. Step 2: a Writer agent and node; the Supervisor's routing now ends with a call to the Writer instead of `FINISH` directly. **What stays the same:** the Notes Agent's and Web Agent's own logic — untouched since Step 1, still called the same way through their Step 2 node wrappers.

When you'll hit this for real: any pipeline where multiple sources feed into one final deliverable — a report, a summary, an answer — that a person or downstream system actually reads; raw findings from separate sources almost always need a synthesis step, not just concatenation. **Helpful**

background: the Generator → Critic → Revision multi-agent pattern, for the synthesis-role idea; LCEL — connecting pieces with `|` — since the Writer is a plain chain, the same shape as any plain LLM-chain Writer agent.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `writer_agent.py`: `run_writer(notes_findings: str, web_findings: str) -> str`, a plain `prompt | ChatOpenAI | output_parser` chain (the usual config/logging setup, no tools, no MCP connection at all). Write the prompt to explicitly instruct synthesis, not concatenation — something like "combine these findings into one short report; don't just restate each source separately." 2. Add `writer_agent_node(state: TeamState) -> Command`, which reads `notes_findings` and `web_findings` from state, calls `run_writer`, writes the result into a new `report: str` field, and returns a `Command` routing to `END`. 3. Update `supervisor.py` so that once every specialist the task actually needed has contributed (check your routing logic from Step 2 against `state["next_agent"]` or a small `completed` tracking field), it routes to `"writer_agent"` instead of straight to `"FINISH"`. 4. Update `graph.py` to add the Writer node, and update `main.py` to run all 4 tasks from **A Real Example** except task 4 (that's Step 4's test) — confirm task 3 produces one combined report mentioning both the notes content and the fetched page content, not two disconnected paragraphs.

Your files after Step 3:

```

project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py
├─ state.py                → adds report: str
├─ supervisor.py          → now routes to writer_agent once specialists are d
├─ notes_agent.py
├─ web_agent.py
├─ writer_agent.py        → run_writer(notes_findings, web_findings) -> str
├─ graph.py               → adds the writer node
└─ main.py                → runs the full team, prints the final synthes

```

Step 4 — Graceful Degradation When One MCP Server Is Down

Project: MCP Crew Multi-Agent Research Team Powered By MCP Servers — Step 4 of 4: Graceful Degradation When One MCP Server Is Down

What this step does: makes the whole team survive the Web Agent's MCP server being unreachable — the Supervisor should still produce a report using just the Notes Agent's findings, with the report clearly saying the web check couldn't be done, instead of the whole run crashing.

Why this step matters: this is the same graceful-degradation lesson from single-agent MCP work — a dead MCP server shouldn't crash everything depending on it — proven again one level up, at the team level instead of the single-agent level. A team is less trustworthy than a single agent if it's *more* fragile, not less, when one piece goes down. **What's new vs. Step 3:** a timeout +

`try / except` around the Web Agent's connection attempt, a `web_unavailable: bool` field in shared state, and a Writer prompt that reads that flag and says so in the report. **What stays the same:** the Notes Agent, and the Web Agent's behavior when its server *is* reachable — resilience wraps around the existing team, it doesn't rewrite it. **When you'll hit this for real:** the first time you demo this project (or any multi-agent system with external connections) to someone else and one of your servers isn't running yet — which will happen, and is exactly why this step exists before you call the project done. **Helpful background:** common single-agent failure modes, worth naming now; how errors propagate between agents in a multi-agent system.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. In `mcp_connection.py`, wrap the connect-plus-`initialize()` call in `asyncio.wait_for(..., timeout=...)`. In `web_agent.py`'s `run_web_agent`, catch a timeout or connection failure with `try / except`, log it clearly (`"[web_agent] server unreachable: <reason>, continuing without it"`), and return a clear sentinel string (like `"WEB_UNAVAILABLE"`) instead of letting the exception escape. 2. Update `web_agent_node` so that when it sees that sentinel, it sets `state["web_unavailable"] = True` and writes an empty or clearly-labeled `web_findings`, instead of crashing the node. 3. Update `supervisor.py` so it still

routes to `writer_agent` after a failed Web Agent call — a missing web server is not a reason to end the run early, it's a reason to finish with less information. 4. Update `writer_agent.py`'s prompt-building so that when `web_unavailable` is `True`, the prompt tells the model plainly that the web check could not be completed, and instructs it to say so in the report rather than silently omitting it or inventing something. 5. Write `test_degraded_run.py`: run task 3 from **A Real Example** twice — once with both servers up (confirm a full report using both sources), once with `web_server.py` deliberately never started (confirm the run still finishes, using only the Notes Agent's findings, and the final report clearly states the web page couldn't be reached).

Your files after Step 4 (final):

```
project_11_mcpcrew_multi_agent_mcp/
├─ config.py
├─ logging_setup.py
├─ notes_server.py
├─ web_server.py
├─ mcp_connection.py          → connect wrapped in a timeout + try/except; f
├─ state.py                  → adds web_unavailable: bool
├─ supervisor.py             → still routes to writer_agent after a fai
├─ notes_agent.py
├─ web_agent.py              → catches an unreachable server, sets w
├─ writer_agent.py           → prompt explicitly flags missing we
├─ graph.py
├─ main.py                   → full demo: runs with both server
└─ test_degraded_run.py      → confirms the team finishes an
```

Final Deliverable: MCP Crew Multi-Agent Research Team Powered By MCP Servers — a Supervisor-led team of three agents, where the Notes Agent and Web Agent each get their tools from their own dedicated MCP server instead of hardcoded Python, the Writer synthesizes both specialists' findings into one coherent report, and the whole team keeps working — with a clear, honest gap noted — when one specialist's MCP server is unreachable.

Checklist Before You Call This Done

- [] The Notes Agent and Web Agent each connect to their own separate MCP server — no tool is hardcoded directly into either specialist's own Python code
- [] Each specialist was proven working completely alone (Step 1) before the Supervisor (Step 2) ever routed to it
- [] The Supervisor routes using the `Command` tool — not a hand-made text-code router
- [] A test where the Supervisor calls both specialists in one run, and the Writer's final report genuinely combines both, not just concatenates them
- [] Shared state only holds each specialist's *finished* result, not its private scratchpad or tool-call history

- [] A test where the Web Agent's MCP server is deliberately not running — the team still finishes, using only the Notes Agent's findings, and the final report clearly says the web check couldn't be done
- [] You can explain, for each specialist, exactly which piece of code owns its MCP connection, and confirm the Supervisor never touches that connection directly

Status

Not started. Track your own progress however works for you.